

Algoritmos e Estrutura de Dados 1

TAD - Tipos Abstratos de Dados

Um tipo abstrato é aquele que não define a forma como será implementado, mas sim, o **modelo** que representa e as **operações** possíveis.

- O TAD não define a estratégia de implementação;
- Na implementação, vamos usar estruturas de dados, como **arrays e structs**, para construir o TAD.

Vamos conhecer os seguintes TADs:

- **Lista Sequencial**
- **Lista Encadeada**
- **Lista Duplamente Encadeada**
- **Lista Circular**
- **Fila**
- **Pilha**

Lista Sequencial (Array Dinâmico)

Sequência de dados armazenados contiguamente.

Permite acesso sequencial e aleatório aos seus elementos.

- **Elementos são acessados por índice (int);**

Operações típicas:

- **Criar e Destruir;**
- **Adicionar, Inserir e Remover;**
- **Acessar e Pesquisar.**

Size: 6

5	9	15	5	7	1
0	1	2	3	4	5

- Características:
 - **Acesso sequencial e aleatório;**
 - **Bom para para inserções/remoções ao final da lista;**
 - **Ruim para inserções/remoções no início ou meio da lista.**

```
lista = create()
lista.add(5)
lista.add(9)
lista.add(15)
lista.add(4)
lista.insert(1, 7)
```

size: 5

5	7	9	15	4
0	1	2	3	4

- Implementada clássica utiliza vetor
 - Vetor redimensionável, alocado em **Heap. Class** :
 - Capacidade do vetor → tamanho real do vetor
 - Tamanho da lista → quantidade de elementos na lista
 - Ponteiro para o vetor alocado em heap

```
class ListSeq {
public:
    int size = 0;    // número de posições ocupadas
    int capacity;    // capacidade da lista (tamanho do vetor)
    int* data;       // endereço do vetor na heap
};
```

Vantagens/Desvantagens

Característica	Lista Sequencial
Acesso sequencial	sim
Acesso aleatório	sim
Redimensionamento	custoso
Inserção/remoção no início	custoso
Inserção/remoção no final	fácil
Inserção/remoção no meio	custoso

Exemplos: (Maior vantagem é o acesso aleatório!)

1. Catálogo de produtos em um e-commerce com número fixo de categorias;
2. Grades de alunos em uma turma com número determinado de alunos;
3. Armazenar as temperaturas médias de cada dia do mês (sempre 30/31 posições);
4. Sistema de reservas de assentos em um cinema (acesso direto ao assento 15F).

Em geral seriam casos que não são precisos inserções/remoções no início e meio ou que precisam de acesso aleatório.

Lista Sequencial - Operações

- Manipulação da Lista Sequencial ocorre por meio das operações

```
ListSeq ls(5);
ls.add(5);    // adiciona no final
ls.add(9);
ls.add(15);
ls.add(7);
ls.remove();  // remove no final
ls.print();   // mostra elementos
```

Operações de criação e acesso

```
// construtor que cria a instância da lista
ListSeq(int _capacity);
// Libera a memória alocada para o vetor
void destroy();
// realoca vetor
bool resize();
// imprime conteúdo da lista
bool print();
// retorna posição do elemento ou -1
int find(int elem);
// obtém o elemento na posição passada
int get(int pos);
// lista vazia?
bool isEmpty();
// lista cheia?
bool isFull();
```

```
ListSeq::ListSeq(int _capacity)
{
    data = new int[_capacity]; // new sempre aloca na heap
    capacity = _capacity;
};

void ListSeq::destroy()
{
    delete[] data;
}

bool ListSeq::resize()
{
    int *_data = new int[++capacity]; // Um novo array dinâmico _data é criado com tamanho capacity + 1.
    // O operador ++capacity incrementa o valor de capacity antes de usá-lo, aumentando a capacidade da lista em 1.
    // Isso significa que a lista terá espaço para armazenar mais um elemento.

    for (int i = 0; i < size; i++)
    {
        _data[i] = data[i];
    }
}
```

```
        // O loop copia os elementos existentes da lista original (data)
para o novo array (_data).
        // Isso garante que os elementos já armazenados na lista sejam
mantidos após o redimensionamento.

        delete[] data; // semelhante ao free(), apaga tudo
data = _data;

        return true;
};

void ListSeq::print()
{
    for (int i = 0; i < size; i++)
    {
        cout << data[i] << " ";
    }

    cout << endl; // mesma coisa que o \n
};

int ListSeq::find(int elem)
{
    for (int i = 0; i < size; i++)
    {
        if (elem == data[i])
        {
            return i;
        }
    }
    return -1;
}

int ListSeq::get(int pos)
{
    if (pos >= size)
    {
        return -1;
    }

    return data[pos];
}
```

```

bool ListSeq::isEmpty()
{
    return size == 0;
};

bool ListSeq::isFull()
{
    return size == capacity;
};

```

Operações de modificação

```

// insere elemento no final
bool add(int elem);
// remove elemento do final
bool remove();
// insere elemento na posição especificada
void insert(int elem, int pos);
// remove elemento na posição especificada
bool removeAt(int pos);
// insere elemento em ordem crescente
bool addSorted(int elem);

```

```

bool ListSeq::add(int elem)
{
    if (size < capacity) // Verifica se há espaço disponível na lista
    {
        data[size++] = elem; // Adiciona o elemento na posição atual de
        'size' e incrementa 'size'
        return true; // Retorna true indicando que o elemento
        foi adicionado com sucesso
    }
    else
    {
        return false; // Retorna false se a lista estiver cheia
    }
};

```

```
void ListSeq::remove()
{
    if (!isEmpty()) // Verifica se a lista não está vazia
    {
        size--; // Decrementa o tamanho da lista, "removendo" o último elemento
    }
};

void ListSeq::insert(int elem, int pos)
{
    if (pos < size) // Verifica se a posição é válida
    {
        for (int i = size; i > pos; i--)
        {
            data[i] = data[i - 1]; // Move os elementos para a direita
        }
        data[pos] = elem; // Insere o novo elemento na posição desejada
        size++;          // Incrementa o tamanho da lista
    }
    else if (pos == size) // Se a posição for igual ao tamanho atual da lista
    {
        add(elem); // Adiciona o elemento ao final da lista
    }
    else // Caso 3: Posição inválida (fora do intervalo)
    {
        return;
    }
}
```

```

void ListSeq::removeAt(int pos)
{
    if (pos < size) // Caso 1: Posição válida dentro do intervalo
    {
        for (int i = pos; i < size - 1; i++)
            // O loop for começa na posição pos e continua até o penúltimo
            elemento (size - 1).
            // Ele copia o próximo elemento (i + 1) para a posição atual
            (i), efetivamente "removendo" o elemento na posição pos.
        {
            data[i] = data[i + 1]; // Copia o próximo elemento para a
            posição atual
        }
        size--; // Decrementa o tamanho da lista
    }

    else if (pos == size - 1) // Caso 2: Posição é o último elemento
    {
        remove(); // Reutiliza o método remove() para remover o último
        elemento
    }

    else // Caso 3: Posição inválida (fora do intervalo)
    {
        return;
    }
};

bool ListSeq::addSorted(int elem)
// Adiciona um elemento à lista de forma ordenada
{
    if (isFull()) // Verifica se a lista está cheia
    {
        return false; // Retorna false se a lista estiver cheia
    }

    // Ordena a lista antes de inserir o elemento
    std::sort(data, data + size);

    // A função std::sort ordena os elementos no intervalo [begin, end)
    em ordem crescente.
    // O intervalo é definido por dois iteradores: begin (início) e end
    (fim).

```

```

    // data é o ponteiro para o início do array que contém os elementos
da lista.
    // data + size é o ponteiro para a posição logo após o último
elemento da lista.

    // Encontra a posição correta para inserir o elemento
    int i = 0;
    while (i < size && data[i] < elem)
    {
        i++;
    }

    // Insere o elemento na posição encontrada
    insert(elem, i);

    return true; // Retorna true indicando que o elemento foi
adicionado com sucesso
}

```

Lista Encadeada

Encadeamento de nós que armazenam dados:

- Acesso sequencial;
- Não permite acesso aleatório;
- **Bom para inserções/remoções em qualquer parte da lista;**
- Disponível em C++ pela STL como `std::forward_list`
- `std::List` é a Lista Duplamente Encadeada

•

```
head -> |3| -> |8| -> |2| -> |5| -> __
```

Nodes

```
class Node {  
public:  
    int key;  
    Node* next;  
};  
  
class LinkedList {  
private:  
    Node* head;  
...}
```

```
head -> |3|    ->    |8| -> |2| -> |5| -> --  
      Elem  Next
```

Vantagens/Desvantagens

Característica	Lista Sequencial	Lista Encadeada
Acesso sequencial	sim	sim
Acesso aleatório	sim	não
Redimensionamento	custoso	fácil
Inserção/remoção no início	custoso	fácil
Inserção/remoção no final	fácil	fácil (c/ endereço)
Inserção/remoção no meio	custoso	fácil (c/ endereço)

Importante falar também que ela gasta mais memória, pois utiliza ponteiros!

Exemplos:

1. Editor de texto (como o Notepad)

- Cada linha do texto pode ser um nó numa lista encadeada. Assim, inserir ou deletar linhas é rápido e não precisa mover todo o texto seguinte.

1. Gerenciamento de memória (heap)

- Muitos sistemas de gerenciamento de memória (malloc/free) usam listas encadeadas para controlar blocos livres ou alocados.

1. Programas que processam músicas/filmes em listas de reprodução

- Um reprodutor de música pode ter uma playlist que você altera dinamicamente (adiciona/remove faixas), sem se preocupar em reorganizar arrays.

Resumindo, quando for preciso fazer inserções/remoções rápidas no meio ou no início. Ou quando o tamanho é muito variável e você não quer ficar se preocupando com o redimensionamento.

Lista Encadeada - Cabeçalhos

```
class LinkedList {  
private:  
    Node* head;  
public:  
    LinkedList();  
    ~LinkedList();  
    void push_front(int key);  
    bool pop_front();  
    int get(int pos);  
    void print();  
    int size();  
    bool empty();  
    void push_back(int key);  
    bool pop_back();  
};
```

```
LinkedList::LinkedList()  
{  
    this->head = nullptr;  
}
```

```
LinkedList::~~LinkedList()  
{  
    Node *aux = this->head; // Ponteiro auxiliar para percorrer a lista  
    while (aux) // Enquanto houver nós na lista  
    {  
        Node *temp = aux; // Armazena o nó atual  
        aux = aux->next; // Avança para o próximo nó  
        delete temp; // Libera a memória do nó atual  
    }  
    this->head = nullptr; // Atualiza o ponteiro "head" para nullptr,  
indicando que a lista está vazia  
}
```

```

bool LinkedList::push_front(int key)
{ // Adiciona um nó no início da lista
    Node *no = new Node{key, nullptr};
    if (!no) // Verifica se a alocação de memória foi bem-sucedida
    {
        return false;
    }
    no->next = this->head; // O ponteiro "next" do novo nó aponta para
o nó atual "head"
    this->head = no;      // O ponteiro "head" agora aponta para o
novo nó, tornando-o o primeiro nó da lista
    return true;          // Retorna true indicando que o nó foi
adicionado com sucesso
}

bool LinkedList::pop_front()
{
    if (this->head) // Verifica se a lista não está vazia
        // Se a lista não estiver vazia, remove o primeiro nó
        {
            Node *aux = this->head; // Ponteiro auxiliar para armazenar o
nó a ser removido
            this->head = aux->next; // O ponteiro "head" agora aponta para
o segundo nó da lista
            delete aux; // Libera a memória do nó removido
            return true;
        }
    else
    {
        return false;
    }
}

int LinkedList::get(int pos)
{
    Node *aux = this->head;
    if (pos < 0 || pos >= this->size()) // Verifica se a posição é
válida
    {
        return 0; // Retorna 0 se a posição for inválida
    }
    for (int i = 0; i < pos; i++) // Percorre a lista até a posição
desejada

```

```

    {
        aux = aux->next;
    }
    return aux->key; // Retorna a chave do nó na posição desejada
}

void LinkedList::print()
{
    Node *aux = this->head;
    while (aux)
    {
        cout << aux->key << "->";
        aux = aux->next;
    }

    cout << endl;
}

int LinkedList::size()
{
    int n = 0;
    Node *aux = this->head;
    while (aux) // Percorre a lista contando o número de nós
    {
        aux = aux->next;
        n++;
    }
    return n;
}

bool LinkedList::empty()
{
    if (this->head == nullptr) // Verifica se o ponteiro "head" é nulo,
    indicando que a lista está vazia
    {
        return true; // Retorna true se a lista estiver vazia
    }

    return false; // Retorna false se a lista não estiver vazia
}

```

```
bool LinkedList::push_back(int key)
{
    Node *no = new Node{key, nullptr};
    if (!no) // Verifica se a alocação de memória foi bem-sucedida
    {
        return false;
    }
    if (!this->head) // Se a lista estiver vazia, o novo nó se torna o  
primeiro nó
    {
        this->head = no;
    }
    else
    {
        Node *aux = this->head; // Ponteiro auxiliar para percorrer a  
lista
        while (aux->next) // Percorre até o último nó da lista
        {
            aux = aux->next;
        }
        aux->next = no; // O ponteiro "next" do último nó aponta para o  
novo nó
    }
    return true;
}
```

```

bool LinkedList::pop_back()
{
    if (!this->head)
    {
        return false; // Retorna false se a lista estiver vazia
    }

    if (!this->head->next) // Caso especial: apenas um nó na lista ||
um if (this->size() == 1) também funcionaria
    {
        delete this->head;
        this->head = nullptr; // Atualiza o ponteiro "head" para
nullptr
        return true;
    }

    Node *aux = this->head;

    while (aux->next->next) // Percorre a lista até o penúltimo nó ||
Um while(this->size() - 1) também funcionaria
    {
        aux = aux->next;
    }

    delete aux->next; // Libera a memória do último nó
    aux->next = nullptr; // O ponteiro "next" do penúltimo nó agora
aponta para nullptr, removendo o último nó da lista
    return true; // Retorna true indicando que o nó foi removido com
sucesso
}

```

Lista Encadeada - Outras operações

```
Node* find(int key);
bool insert_after(int key, Node* pos);
bool remove_after(Node* pos);
bool insert(int pos);
bool remove(int pos);
bool remove(int key);
bool insert_sorted(int key);
```

```
Node* LinkedList::find(int key)
{
    Node* aux = this->head;

    while (aux) // Percorre a lista enquanto houver nós
    {
        if (aux->key == key) // Verifica se o nó atual contém a chave
        {
            return aux; // Retorna o nó encontrado
        }
        aux = aux->next; // Avança para o próximo nó
    }

    return nullptr; // Retorna nullptr se a chave não for encontrada
}
```

```

bool LinkedList::insert(int key, int pos)
{
    if (pos < 0 || pos > this->size()) // Verifica se a posição é
válida
    {
        return false; // Retorna false se a posição for inválida
    }

    if (pos == 0) // Insere no início da lista
    {
        return this->push_front(key);
    }

    Node *aux = this->head;

    for (int i = 0; i < pos - 1 && aux; i++) // Percorre a lista até o
nó anterior à posição desejada
        //O loop precisa parar no nó anterior à posição pos para que o novo
nó possa ser inserido corretamente.
        //O loop também verifica se aux não é nulo, garantindo que não
acesse um ponteiro nulo.
    {
        aux = aux->next;
    }

    Node *novo = new Node{key, aux->next}; // Cria um novo nó com a
chave e o ponteiro "next" apontando para o próximo nó
    if (!novo) // Verifica se a alocação de memória foi bem-sucedida
    {
        return false;
    }

    aux->next = novo; // O ponteiro "next" do nó anterior agora aponta
para o novo nó
    //O novo nó é inserido na lista, e o ponteiro "next" do nó anterior
agora aponta para o novo nó.

    return true;
}

```

```

Node *LinkedList::removePos(int pos)
{
    if (pos < 0 || pos >= this->size()) // Verifica se a posição é
válida
    {
        return nullptr; // Retorna nullptr se a posição for inválida
    }
    if (pos == 0) // Se a posição for 0, remove o primeiro nó
    {
        pop_front();
        return this->head; // Retorna o novo primeiro nó (ou nullptr se
a lista estiver vazia)
    }

    if (pos == this->size() - 1) // Se a posição for o último nó,
remove o último nó
    {
        pop_back();
        return nullptr; // Retorna nullptr, pois o último nó foi
removido
    }

    Node *aux = this->head;
    for (int i = 0; i < pos - 1 && aux; i++) // Percorre a lista até o
nó anterior à posição desejada
    {
        aux = aux->next;
    }
    Node* temp = aux->next; // Armazena o nó a ser removido
    aux->next = aux->next->next; // Atualiza o ponteiro "next" do nó
anterior para pular o nó a ser removido
    delete temp; // Libera a memória do nó removido
    return aux->next; // Retorna o próximo nó após o nó removido
}

```

```

bool LinkedList::removeKey(int key)
{
    if (!this->head) // Verifica se a lista está vazia
    {
        return false;
    }
    // Caso especial: o nó a ser removido é o primeiro
    if (this->head->key == key)
    {
        Node *temp = this->head; // Armazena o nó a ser removido
        this->head = this->head->next; // Atualiza o ponteiro "head"
para o próximo nó
        delete temp; // Libera a memória do nó removido
        return true;
    }

    Node *aux = this->head;
    // Percorre a lista procurando o nó com a chave
    while (aux->next) // Garante que aux->next não seja nullptr
    {
        if (aux->next->key == key) // Verifica se o próximo nó contém a
chave
        {
            Node *temp = aux->next; // Armazena o nó a ser removido
            aux->next = aux->next->next; // Atualiza o ponteiro "next"
do nó atual para pular o nó a ser removido
            delete temp; // Libera a memória do nó removido
            return true; // Retorna true indicando que o nó foi
removido com sucesso
        }
        aux = aux->next; // Avança para o próximo nó
    }
    return false; // Retorna false se a chave não for encontrada
}

```

Estratégias Avançadas

- Adicionar ponteiro para cauda da lista

- Facilita as **inserções ao final da lista**;
- Acarreta em mais código de controle (inserir e remover).

```
struct LinkedList {  
    Node* head; // primeiro Node  
    Node* tail; // último Node  
};  
  
struct Node {  
    int key;  
    Node* next;  
};
```

-

- Emprego de **sentinela**

- Elemento **"coringa"** como primeiro da lista

```
head -> |*| -> __
```

-

- Simplifica a implementação, pois **"head" nunca precisa ser atualizado:**

- Na implementação tradicional, as operações inserir e remover requerem verificações para atualizar o ponteiro "head".
- Com a sentinela, não há necessidade de atualizar "head"

Lista Duplamente Encadeada

- Cada nó possui dois ponteiros, **prev** e **next** , que apontam, respectivamente, **ao nó anterior e ao próximo**, na lista;
- **Facilita ainda mais inserções/remoções em qualquer parte da lista**;
- Disponível em C++ pela STL como `std::List`.

```
      ---  
      ^  
head -> |3| <-> |8| <-> |2| <-> |5| -> ---
```

- Podemos já deixar implementado as estratégias avançadas:
 - **Ponteiro para a cauda**;
 - **Sentinela**.

```
      ---  
      ^  
head -> |*| <-> |3| <-> |8| <-> |2| <-> |5| -> ---  
                                         ^  
                                         tail
```

Estrutura

```
class Node {  
public:  
    int key;  
    Node* next;  
    Node* prev;  
};
```

```
class List { //ou pode ser DoubleList
private:
    Node* head;
    Node* tail;
```

Vantagens/Desvantagens

Aspecto	Lista Encadeada Simples	Lista Duplamente Encadeada
Direção	Só pra frente	Frente e trás
Memória por Nó	Menor (1 ponteiro)	Maior (2 ponteiros)
Facilidade de Remoção	Pode ser difícil (precisa do anterior)	Fácil
Complexidade de Código	Mais simples	Mais complexo

Exemplo de Uso

1. Navegadores Web
2. Edição de Texto Avançada
3. Jogos de Turnos

Resumindo, usamos a duplamente encadeada quando precisa de **acesso fácil para trás e para frente**, quando **a remoção de qualquer nó precisa ser rápida sem percorrer toda a lista**, quando é comum navegar para ambos os lados (ex: **undo/redo, navegador, cache**).

Métodos

```
DoubleList::DoubleList()
{
    this->head = nullptr;
    this->tail = nullptr;
}
DoubleList::~DoubleList()
{
    Node *aux = this->head;
    while (aux)
```

```

    {
        Node *temp = aux;
        aux = aux->next;
        delete temp;
    }
    this->head = nullptr;
    this->tail = nullptr;
}

bool DoubleList::push_front(int key) {
    Node *new_node = new Node{key, nullptr, nullptr};
    if (!new_node){
        return false; // Verifica se a alocação de memória foi
bem-sucedida
    }
    if (!this->head) // Lista vazia
    {
        this->head = this->tail = new_node; // O novo nó se torna o
primeiro e o último nó da lista
    }
    else
    {
        new_node->next = this->head; // O ponteiro "next" do novo nó
aponta para o nó atual "head"
        this->head->prev = new_node; // O ponteiro "prev" do nó atual
"head" aponta para o novo nó
        this->head = new_node; // O ponteiro "head" agora aponta para o
novo nó, tornando-o o primeiro nó da lista
    }
    return true;
}

bool DoubleList::pop_front() {
    if (!this->head){ // Lista vazia
        return false;
    }
    Node *temp = this->head;
    if (this->head == this->tail) // Apenas um nó na lista
    {
        this->head = this->tail = nullptr; // Ambos os ponteiros "head"
e "tail" são definidos como nullptr
    }
    else

```

```

    {
        this->head = this->head->next; // O ponteiro "head" agora
aponta para o segundo nó da lista
        this->head->prev = nullptr; // O ponteiro "prev" do novo
primeiro nó é definido como nullptr
    }
    delete temp; // Libera a memória do nó removido
    return true;
}

bool DoubleList::push_back(int key) {
    Node *new_node = new Node{key, nullptr, nullptr};
    if (!new_node){
        return false; // Verifica se a alocação de memória foi
bem-sucedida
    }
    if (!this->tail) // Lista vazia
    {
        this->head = this->tail = new_node; // O novo nó se torna o
primeiro e o último nó da lista
    }
    else
    {
        new_node->prev = this->tail; // O ponteiro "prev" do novo nó
aponta para o nó atual "tail"
        this->tail->next = new_node; // O ponteiro "next" do nó atual
"tail" aponta para o novo nó
        this->tail = new_node; // O ponteiro "tail" agora aponta para o
novo nó, tornando-o o último nó da lista
    }
    return true;
}

bool DoubleList::pop_back() {
    if (!this->tail){ // Lista vazia
        return false; // Retorna false se a lista estiver vazia
    }

    Node *temp = this->tail;
    if (this->head == this->tail) // Apenas um nó na lista
    {
        this->head = this->tail = nullptr; // Ambos os ponteiros "head"
e "tail" são definidos como nullptr
    }
}

```

```

    }
    else
    {
        this->tail = this->tail->prev; // O ponteiro "tail" agora
aponta para o penúltimo nó da lista
        this->tail->next = nullptr; // O ponteiro "next" do novo último
nó é definido como nullptr
    }
    delete temp;
    return true;
}

int DoubleList::get(int pos) {
    if (pos < 0 || pos >= this->size()){
        return -1; // Retorna -1 para posição inválida
    }

    Node *aux = this->head;
    for (int i = 0; i < pos; i++)
    {
        aux = aux->next;
    }
    return aux->key;
}

void DoubleList::print() {
    Node *aux = this->head;
    while (aux)
    {
        std::cout << aux->key << " ";
        aux = aux->next;
    }
    std::cout << std::endl;
}

int DoubleList::size() {
    int count = 0;
    Node *aux = this->head;
    while (aux)
    {
        count++;
        aux = aux->next;
    }
}

```

```

        return count;
    }

bool DoubleList::empty() {
    return this->head == nullptr;
}

Node *DoubleList::find(int key)
{
    Node *aux = this->head;
    while (aux)
    {
        if (aux->key == key)
            return aux;
        aux = aux->next;
    }
    return nullptr;
}

bool DoubleList::insert(int key, int pos)
{
    if (pos < 0 || pos > this->size()) // Verifica se a posição é inválida
    {
        return false;
    }

    if (pos == 0) // Inserção no início
    {
        return this->push_front(key);
    }

    if (pos == this->size()) // Inserção no final
    {
        return this->push_back(key);
    }

    Node *aux = this->head;
    for (int i = 0; i < pos - 1; i++) // Percorre até o nó anterior à posição desejada
    {
        aux = aux->next;
    }

```

```

Node *new_node = new Node{key, aux->next, aux}; // Cria o novo nó
if (!new_node)
{
    return false;
}

    aux->next->prev = new_node; // Atualiza o ponteiro "prev" do
próximo nó
    aux->next = new_node;      // Atualiza o ponteiro "next" do nó
anterior

    return true;
}

Node *DoubleList::removePos(int pos)
{
    if (pos < 0 || pos >= this->size()) // Verifica se a posição é
inválida
    {
        return nullptr;
    }

    if (pos == 0) // Remoção do primeiro nó
    {
        Node *removed = this->head;
        this->pop_front();
        return removed;
    }

    if (pos == this->size() - 1) // Remoção do último nó
    {
        Node *removed = this->tail;
        this->pop_back();
        return removed;
    }

    Node *aux = this->head;
    for (int i = 0; i < pos; i++) // Percorre até o nó na posição
desejada
    {
        aux = aux->next;
    }

```

```

        aux->prev->next = aux->next; // Atualiza o ponteiro "next" do nó
anterior
        aux->next->prev = aux->prev; // Atualiza o ponteiro "prev" do
próximo nó

        aux->next = aux->prev = nullptr; // Desconecta o nó removido
        return aux;                      // Retorna o nó removido
    }

bool DoubleList::removeKey(int key)
{
    if (!this->head) // Verifica se a lista está vazia
    {
        return false;
    }

    if (this->head->key == key) // Caso especial: o nó a ser removido é
o primeiro
    {
        this->pop_front();
        return true;
    }

    if (this->tail->key == key) // Caso especial: o nó a ser removido é
o último
    {
        this->pop_back();
        return true;
    }

    Node *aux = this->head;
    while (aux) // Percorre a lista procurando o nó com a chave
    {
        if (aux->key == key)
        {
            aux->prev->next = aux->next; // Atualiza o ponteiro "next"
do nó anterior
            if (aux->next)                // Verifica se não é o último
nó
            {
                aux->next->prev = aux->prev; // Atualiza o ponteiro
"prev" do próximo nó
            }
        }
    }
}

```

```

    }

    delete aux; // Libera a memória do nó removido
    return true;
}

aux = aux->next; // Avança para o próximo nó
}

return false; // Retorna false se a chave não for encontrada
}

```

Lista Circular

- Possui a vantagem de não tratar um ponteiro null (exceto se estiver vazia).



Estrutura

```

class Node {
public:
    int key;
    Node* next;
};

class CircularList {
private:
    Node* head;
    Node* tail;
}

```

Vantagens

- **Navegação contínua:** você pode percorrer toda a lista a partir de qualquer nó sem precisar voltar manualmente ao começo (**não existe "fim de lista"**).
- **Boa para aplicações "cíclicas":** onde você precisa dar voltas em uma coleção de elementos.
- **Inserções/remoções fáceis no início e no final:** você pode inserir rapidamente depois de um nó específico, sem precisar ir até o fim da lista.
- **Economiza verificações:** muitas vezes você não precisa checar se chegou ao final — **basta testar se voltou ao nó inicial**.

Desvantagens

- **Pode ser confuso:** a lógica de parada em percursos deve ser bem cuidada para não entrar em laços infinitos.
- **Acesso ainda é sequencial:** para encontrar um elemento específico, o custo continua sendo $O(n)$.
- **Implementação mais complexa que uma lista "normal":** principalmente na hora de inserir/remover nós, já que tem que manter a "circularidade" certinha.

Exemplos

Resumindo, usamos a lista circular quando **precisamos de uma estrutura que permita iteração contínua entre os elementos**, como em escalonamento de processos, alternância de turnos em jogos, buffers de dados que sobrescrevem automaticamente, navegação cíclica em menus ou listas, **ou quando as operações precisam “voltar ao início” de forma automática e eficiente**.

Aspecto	Lista Linear	Lista Circular
Final da lista	Termina (NULL)	Volta para o início
Percurso	Pode acabar naturalmente	Pode ser infinito se não for controlado
Usado em	Listagens simples	Sistemas cíclicos

Métodos

```

CircularList::CircularList()
{
    this->head = nullptr;
    this->tail = nullptr;
}

CircularList::~~CircularList()
{
    if (!this->head) // Lista vazia
        return;

    Node *aux = this->head;
    while (aux != this->tail) // Percorre até o último nó
    {
        Node *temp = aux;
        aux = aux->next;
        delete temp;
    }

    delete aux; // Libera o último nó
    this->head = nullptr;
    this->tail = nullptr;
}

bool CircularList::push_front(int key)
{
    Node *new_node = new Node{key, nullptr};
    if (!new_node){
        return false; // Verifica se a alocação de memória foi bem-sucedida
    }

    if (!this->head) // Lista vazia

```

```

    {
        this->head = this->tail = new_node; // O novo nó se torna o
head e o tail
        new_node->next = new_node; // Nó aponta para si mesmo
    }
    else
    {
        new_node->next = this->head; // O novo nó aponta para o antigo
head
        this->head = new_node; // O head agora é o novo nó
        this->tail->next = this->head; // Atualiza o tail para apontar
para o novo head
    }
    return true;
}

bool CircularList::pop_front()
{
    if (!this->head) // Lista vazia
        return false;

    Node *temp = this->head;
    if (this->head == this->tail) // Apenas um nó na lista
    {
        this->head = this->tail = nullptr;
    }
    else
    {
        this->head = this->head->next;
        this->tail->next = this->head; // Atualiza o tail para apontar
para o novo head
    }
    delete temp;
    return true;
}

int CircularList::get(int pos)
{
    if (pos < 0 || pos >= this->size()) // Verifica se a posição é
inválida
        return -1; // Retorna -1 para indicar erro

    Node *aux = this->head;

```

```

        for (int i = 0; i < pos; i++) // Corrige o loop para avançar apenas
até a posição desejada
        {
            aux = aux->next;
        }

        return aux->key;
    }

void CircularList::print()
{
    if (!this->head) // Lista vazia
    {
        std::cout << "Lista vazia" << std::endl;
        return;
    }

    Node *aux = this->head;
    while (true)
    {
        std::cout << aux->key << " ";
        aux = aux->next;
        if (aux == this->head) // Verifica se voltou ao início
            break;
    }

    std::cout << std::endl;
}

int CircularList::size()
{
    if (!this->head) // Lista vazia
        return 0;

    int count = 0;
    Node *aux = this->head;
    while (true)
    {
        count++;
        aux = aux->next;
        if (aux == this->head) // Verifica se voltou ao início
            break;
    }
}

```

```

        return count;
    }

bool CircularList::empty()
{
    return this->head == nullptr;
}

bool CircularList::push_back(int key)
{
    Node *new_node = new Node{key, nullptr};
    if (!new_node)
        return false;

    if (!this->head) // Lista vazia
    {
        this->head = this->tail = new_node; // O novo nó se torna o
head e o tail
        new_node->next = new_node; // Nó aponta para si mesmo
    }
    else
    {
        new_node->next = this->head; // O novo nó aponta para o head
        this->tail->next = new_node; // O antigo tail aponta para o
novo nó
        this->tail = new_node; // Atualiza o tail
    }
    return true;
}

bool CircularList::pop_back()
{
    if (!this->head) // Lista vazia
        return false;

    Node *temp = this->tail;
    if (this->head == this->tail) // Apenas um nó na lista
    {
        this->head = this->tail = nullptr;
    }
    else
    {

```

```

        Node *aux = this->head;
        while (aux->next != this->tail) // Percorre até o penúltimo nó
        {
            aux = aux->next;
        }
        aux->next = this->head; // Atualiza o próximo do penúltimo nó
para o head
        this->tail = aux;      // Atualiza o tail
    }
    delete temp;
    return true;
}

Node *CircularList::find(int key)
{
    if (!this->head) // Lista vazia
        return nullptr;

    Node *aux = this->head;
    while (true)
    {
        if (aux->key == key)
            return aux;
        aux = aux->next;
        if (aux == this->head) // Verifica se voltou ao início
            break;
    }

    return nullptr; // Chave não encontrada
}

bool CircularList::insert(int key, int pos)
{
    if (pos < 0 || pos > this->size()) // Verifica se a posição é
inválida
        return false;

    if (pos == 0) // Inserção no início
        return this->push_front(key);

    if (pos == this->size()) // Inserção no final
        return this->push_back(key);
}

```

```

        Node *aux = this->head;
        for (int i = 0; i < pos - 1; i++) // Percorre até o nó anterior à
posição desejada
        {
            aux = aux->next;
        }

        Node *new_node = new Node{key, aux->next};
        if (!new_node)
            return false;

        aux->next = new_node;
        return true;
    }

Node *CircularList::removePos(int pos)
{
    if (pos < 0 || pos >= this->size()) // Verifica se a posição é
inválida
        return nullptr;

    if (pos == 0) // Remoção do primeiro nó
    {
        Node *removed = this->head;
        this->pop_front();
        return removed;
    }

    Node *aux = this->head;
    for (int i = 0; i < pos - 1; i++) // Percorre até o nó anterior à
posição desejada
    {
        aux = aux->next;
    }

    Node *removed = aux->next;
    aux->next = aux->next->next;

    if (removed == this->tail) // Atualiza o tail se o nó removido for
o último
        this->tail = aux;
}

```

```

        return removed;
    }

bool CircularList::removeKey(int key)
{
    if (!this->head) // Lista vazia
        return false;

    if (this->head->key == key) // Caso especial: o nó a ser removido é o primeiro
    {
        this->pop_front();
        return true;
    }

    Node *aux = this->head;
    while (aux->next != this->head) // Percorre até voltar ao início
    {
        if (aux->next->key == key) // Verifica se o próximo nó contém a chave
        {
            Node *temp = aux->next;
            aux->next = aux->next->next;

            if (temp == this->tail) // Atualiza o tail se o nó removido for o último
                this->tail = aux;

            delete temp;
            return true;
        }
        aux = aux->next;
    }

    return false; // Chave não encontrada
}

```


Lista Circular Duplamente Encadeada

- Inserir no fim, basta inserir no anterior do primeiro, eliminando a necessidade de um ponteiro para cauda.

start -> |3| <-> |8| <-> |2| <-> |5|
 ^-----^

-

Estrutura

```
class Node {
public:
    int key;
    Node* next;
    Node* prev;
};

class CircularList {
private:
    Node* head;
    Node* tail;
```

Vantagens

- **Navegação bidirecional contínua:** você pode andar para frente ou para trás, e nunca "sai" da lista.
- **Operações de inserção e remoção eficientes:** uma vez no nó certo, inserir ou remover nós adjacentes é rápido ($O(1)$).
- **Ideal para estruturas cíclicas:** ótimo para sistemas que precisam de "revezamento" entre elementos.
- **Sem necessidade de tratar NULL em percursos:** porque sempre há um próximo e um anterior.

Desvantagens

- **Código ainda mais complexo:** agora você tem que atualizar dois ponteiros (próximo e anterior) e garantir que o círculo se mantenha correto.
- **Maior uso de memória:** cada nó armazena dois ponteiros.
- **Precisa de atenção para percorrer:** sem um critério de parada claro, você pode entrar num loop infinito.

Comparação:

Aspecto	Lista Encadeada Simples	Lista Duplamente Encadeada	Lista Circular Duplamente Encadeada
Direção	Só para frente	Frente e trás	Frente e trás (e circula)
Final da lista	NULL	NULL	Volta ao início
Complexidade de Código	Menor	Média	Maior
Usado em	Listagens simples	"Undo/Redo"	Turnos, loops infinitos

Exemplos

Resumindo, usamos a lista **circular duplamente encadeada** quando precisamos de iteração contínua **em ambas as direções**, com acesso eficiente tanto para frente quanto para trás, **sem precisar voltar ao início manualmente**, útil em situações como **escalonamento cíclico de processos**, **navegação de menus que pode ir e voltar sem restrição**, **implementação de listas musicais** ou caches circulares onde a ordem de acesso anterior e seguinte importa.

Métodos

```
DoubleCircularList::DoubleCircularList()
{
    this->head = nullptr;
}

DoubleCircularList::~~DoubleCircularList()
{
    if (!this->head) // Lista vazia
        return;
}
```

```

Node *aux = this->head;
while (aux->next != this->head) // Percorre até o último nó
{
    Node *temp = aux;
    aux = aux->next;
    delete temp;
}

delete aux; // Libera o último nó
this->head = nullptr;
}

```

```

bool DoubleCircularList::push_back(int key)
{
    Node *new_node = new Node{key, nullptr, nullptr};
    if (!new_node)
        return false;

    if (!this->head) // Lista vazia
    {
        new_node->next = new_node->prev = new_node; // Nó aponta para
        si mesmo
        this->head = this->tail = new_node; // O head agora é o novo nó
    }
    else
    {
        Node *tail = this->head->prev; // Último nó
    }
}

```

```

        new_node->next = this->head;
        new_node->prev = tail;
        tail->next = new_node;
        this->head->prev = new_node;
    }
    return true;
}

bool DoubleCircularList::pop_front()
{
    if (!this->head) // Lista vazia
        return false;

    Node *temp = this->head;
    if (this->head->next == this->head) // Apenas um nó na lista
    {
        this->head = this->tail = nullptr;
    }
    else
    {
        Node *tail = this->head->prev; // Último nó
        this->head = this->head->next;
        this->head->prev = tail;
        tail->next = this->head;
    }
    delete temp;
    return true;
}

bool DoubleCircularList::pop_back()
{
    if (!this->head) // Lista vazia
        return false;

    Node *tail = this->head->prev;
    if (this->head->next == this->head) // Apenas um nó na lista
    {
        this->head = nullptr;
    }
    else
    {
        Node *new_tail = tail->prev;
        new_tail->next = this->head;
    }
}

```

```

        this->head->prev = new_tail;
    }
    delete tail;
    return true;
}

int DoubleCircularList::get(int pos)
{
    if (pos < 0 || pos >= this->size()) // Verifica se a posição é
    inválida
        return -1;

    Node *aux = this->head;
    for (int i = 0; i < pos; i++)
    {
        aux = aux->next;
    }

    return aux->key;
}

int DoubleCircularList::size()
{
    if (!this->head) // Lista vazia
        return 0;

    int count = 0;
    Node *aux = this->head;
    while (true)
    {
        count++;
        aux = aux->next;
        if (aux == this->head) // Verifica se voltou ao início
            break;
    }

    return count;
}

bool DoubleCircularList::empty()
{
    return this->head == nullptr;
}

```

```

Node *DoubleCircularList::find(int key)
{
    if (!this->head) // Lista vazia
        return nullptr;

    Node *aux = this->head;
    while (true)
    {
        if (aux->key == key)
            return aux;
        aux = aux->next;
        if (aux == this->head) // Verifica se voltou ao início
            break;
    }

    return nullptr; // Chave não encontrada
}

bool DoubleCircularList::insert(int key, int pos)
{
    if (pos < 0 || pos > this->size()) // Verifica se a posição é inválida
        return false;

    if (pos == 0) // Inserção no início
        return this->push_front(key);

    if (pos == this->size()) // Inserção no final
        return this->push_back(key);

    Node *aux = this->head;
    for (int i = 0; i < pos - 1; i++)
    {
        aux = aux->next;
    }

    Node *new_node = new Node{key, aux->next, aux};
    if (!new_node)
        return false;

    aux->next->prev = new_node;
    aux->next = new_node;
}

```

```

        return true;
    }

Node *DoubleCircularList::removePos(int pos)
{
    if (pos < 0 || pos >= this->size()) // Verifica se a posição é
    inválida
        return nullptr;

    if (pos == 0) // Remoção do primeiro nó
    {
        Node *removed = this->head;
        this->pop_front();
        return removed;
    }

    Node *aux = this->head;
    for (int i = 0; i < pos; i++)
    {
        aux = aux->next;
    }

    aux->prev->next = aux->next;
    aux->next->prev = aux->prev;

    return aux;
}

bool DoubleCircularList::removeKey(int key)
{
    if (!this->head) // Lista vazia
        return false;

    if (this->head->key == key) // Caso especial: o nó a ser removido é
    o primeiro
    {
        this->pop_front();
        return true;
    }

    Node *aux = this->head;
    while (true)

```

```

    {
        if (aux->key == key)
        {
            aux->prev->next = aux->next;
            aux->next->prev = aux->prev;
            delete aux;
            return true;
        }
        aux = aux->next;
        if (aux == this->head) // Verifica se voltou ao início
            break;
    }

    return false; // Chave não encontrada
}

void DoubleCircularList::print()
{
    if (!this->head) // Lista vazia
    {
        std::cout << "Lista vazia" << std::endl;
        return;
    }

    Node *aux = this->head;
    while (true)
    {
        std::cout << aux->key << " "; // Imprime o valor do nó atual
        aux = aux->next;                // Avança para o próximo nó
        if (aux == this->head)          // Verifica se voltou ao início
            break;
    }

    std::cout << std::endl; // Quebra de linha após imprimir todos os
    elementos
}

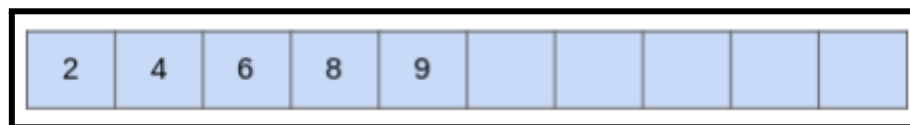
```

STL: Standard Template Library

- Biblioteca padrão de C++ que provê vários TADs

TAD	Classe STL
Lista sequencial	vector
Lista encadeada	forward_list
Lista duplamente encadeada	list
Pilha	stack
Fila	queue

Classe Vector - Lista Sequencial



Operações

```
int main() {  
    vector<int> vec1 = {2,4,6,8,9};    // inicialização estilo array  
    vector<int> vec2;                  // é possível criar lista vazia  
    vector<int> copy = vec1;           // atribuição cria cópia profunda  
  
    for (int i = 1; i <= 5; i++) {  
        vec2.push_back(i*10);          // adiciona 5 elementos ao final da lista  
    }  
    vec2.pop_back();                   // remove o último  
  
    for (int e : vec1)                  // ranged-based for loop C++ ("foreach")  
        cout << e << ' ' ;  
    cout << '\n';  
  
    return 0;  
}
```

vec1<int>

vec2<int>

copy<int>

2	4	6	8	9					
10	20	30	40	50					
2	4	6	8	9					

```

int main() {
    vector<int> vec1 = {2,4,6,8,9};           // inicialização estilo array
    vector<int> copy = vec1;                 // atribuição cria cópia profunda

    copy.insert(copy.begin(), {50, 60});     // insere 50 e 60 no início da lista

    for (int i = 0; i < vec1.size(); i++) // impressão usando laço por índice
        cout << vec1[i] << ' ';
    cout << '\n';

    return 0;
}

```

vec1<int>

2	4	6	8	9					
50	60	2	4	6	8	9			

copy<int>

Formas de Percorrer

```

vector<int> vec = {1,2,4,5,6,7,8};
for (int i = 0; i < vec.size(); i++) // laço de contagem e acesso via índice
    cout << vec[i] << ' ';
cout << endl;

for (int e : vec) // for-each acessando cada elemento
    cout << e << ' ';
cout << endl;

// Iterador passando por cada elemento:
//     vec.end() - retorna um iterador para o final da lista (após último)
//     *it       - operador * permite acessar elemento "apontado" pelo iterador
for (auto it = vec.begin(); it != vec.end(); it++) {
    cout << *it << ' ';
}
cout << endl;

```

Passando para Funções

- Podemos usar passagem por valor ou referência
 - ⇒ valor: não altera vector passado com argumento na chamada
 - ⇔ referência: pode alterar vector passado como argumento na chamada

```
// vec é referência para o vector passado como argumento na chamada da função
void set_vector(vector<int>& vec) {
    for (int i = 0; i < vec.size(); i++)
        vec[i] = i+1;    // altera o vector passado como argumento à função
}
```

```
// vec recebe uma cópia do vector passado como argumento na chamada da função
void set_vector(vector<int> vec) {
    for (int i = 0; i < vec.size(); i++)
        vec[i] = i+1;    // altera somente a cópia local à set_vector()
}
```

- Exemplo de uso de iterador em função que percorre e imprime o conteúdo de um vector (Lista Sequencial)
 - Iteradores permite percorrer e acessar cada elemento da lista

```
void print_vector(vector<int>& vec) {
    for (auto it = vec.begin(); it != vec.end(); it++) {
        cout << *it << ' ';
    }
    cout << '\n';
}
```

Percorrendo o vector com iterador

```
void print_list(vector<int>& vec) {
    for (int e : vec) {
        cout << e << ' ';
    }
    cout << '\n';
}
```

Percorrendo o vector com laço "for-each"

Alocação

- Em C/C++, instâncias de classes/structs podem ser alocados tanto na **stack** (dentro do frame da função) quanto na **heap** (área de alocação livre)
 - Independente do tipo de alocação **vector (Lista Sequencial)**, o array interno é sempre alocado na heap, permitindo aumentar sua capacidade dinamicamente.

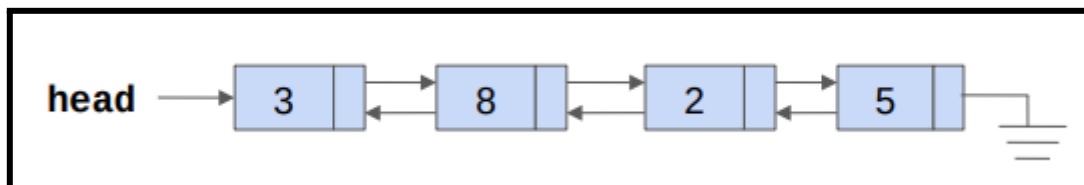
```
// alocação automática (stack frame)
vector<int> vec1 = {1,3,5,7,9};
// alocação manual (heap)
vector<int>* vec2 = new vector<int>({2,4,6,8});

vec1.push_back(55);    // adiciona elemento ao final
vec2->push_back(55);
...
...
delete vec2;    // se usar alocação manual (heap), precisa liberar
```

Resumo

- Principais operações
 - Criar vazia `vector<int> vec`
 - Criar com elementos `vector<int> vec = {3, 6, 8, 9}`
 - Copiar listas `vec2 = vec1`
 - Acessar elemento `vec[índice]`
 - Obter tamanho `vec.size()`
 - Adicionar ao final `vec.push_back(valor)`
 - Remover do final `vec.pop_back()`
 - Inserir elemento `vec.insert(iterador, valor)`
(na posição do iterador)
- Iteradores
 - Obter iterador para início `auto it = vec.begin()`
 - Obter iterador para final (após último) `auto it = vec.end()`
 - Avançar iterador para próximo elemento `it++`
 - Retroceder iterador para elemento anterior `it--`
 - Acessar elemento na posição do iterador `*it`

Classe List - Lista Duplamente Encadeada



Operações

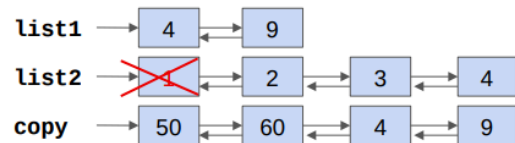
```
#include <iostream>
#include <list>
using namespace std;

int main() {
    list<int> list1 = {4,9};    // inicialização com array
    list<int> list2;           // é possível criar vazia e adicionar elementos
    list<int> copy = list1;     // atribuição cria cópia profunda

    copy.insert(copy.begin(), {50, 60}); // insere 50 e 60 no início da lista

    for (int i = 4; i > 0; i--)
        list2.push_front(i);    // adiciona no início
    list2.pop_front();           // remove do início

    for (int e : list1)
        cout << e << ' ';
    cout << '\n';
    return 0;
}
```



Formas de Percorrer

```
vector<int> vec = {1,2,4,5,6,7,8};    // Uma Lista Encadeada não permite
for (int i = 0; i < vec.size(); i++)   // acesso aleatório aos seus elementos.
    cout << vec[i] << ' ';           // Dessa forma, a list não suporta o
cout << endl;                          // operador de índice: [ ]

for (int e : list1)                    // for-each acessando cada elemento
    cout << e << ' ';

// Iterador passando por cada elemento:
// list1.end() - retorna um iterador para o final da lista (após último)
// *it         - operador * acessa elemento "apontado" pelo iterador
for (auto it = list1.begin(); it != list1.end(); it++) {
    cout << *it << ' ';
}
cout << endl;
```

Funções para Imprimir

- Por conta da padronização dos iteradores da STL e o for-each, a implementação de `print_list()` é idêntica à `print_vector()` da `vector`.

```
void print_list(list<int>& list1) {  
    for (auto it = list1.begin(); it != list1.end(); it++) {  
        cout << *it << ' '  
    }  
    cout << '\n';  
}
```

Percorrendo a list com iterador

```
void print_list(list<int>& list1) {  
    for (int e : list1) {  
        cout << e << ' '  
    }  
    cout << '\n';  
}
```

Percorrendo a list com laço
"for-each"

Iteradores

- Iterador: objeto que aponta para algum elemento em um coleção
 - Possui a habilidade de iterar (navegar) sobre os elementos (++)
 - Acesso ao conteúdo com operador *

```
list<int> list1 = {1,2,3,4,5,6,4,7,8};
```

```
// insere 99 na posição do elemento 4
```

```
for (auto it = list1.begin(); it != list1.end(); it++) {  
    if (*it == 4) {  
        list1.insert(it, 99); // {1,2,3,99,4,5,6,4,7,8}  
                               //           ^  
        break;  
    }  
}
```

// Apaga o primeiro 3 da lista

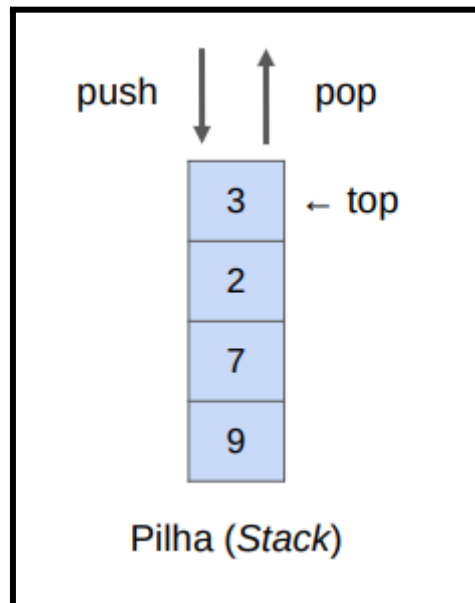
```
for (auto it = list1.begin(); it != list1.end(); it++) {
    if (*it == 3) {
        list1.erase(it); // {1,2,3,99,4,5,6,4,7,8}
                        //      ^
        break;
    }
}

// algumas outras funções (também disponíveis para vector)
list1.remove(4);          // remove todos os elementos 4
list1.reverse();          // inverte a ordem dos elementos na lista
```

Resumo

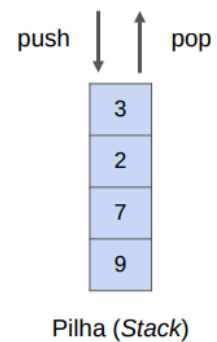
- Principais operações
 - Criar vazia `list<int> list1`
 - Criar com elementos `list<int> list1 = {3, 6, 8, 9}`
 - Copiar listas `list1 = list1`
 - Adicionar no início `list1.push_front(valor)`
 - Remover do início `list1.pop_front()`
 - Adicionar ao final `list1.push_back(valor)`
 - Remover do final `list1.pop_back()`
 - Inserir elemento `list1.insert(iterador, valor)`
(na posição do iterador)
- Iteradores
 - Obter iterador para início `auto it = list1.begin()`
 - Obter iterador para final (após último) `auto it = list1.end()`
 - Avançar iterador para próximo elemento `it++`
 - Retroceder iterador para elemento anterior `it--`
 - Acessar elemento na posição do iterador `*it`

Classe Stack - Pilha



Pilha de Elementos

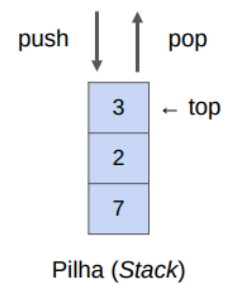
- Pilha (stack) e Fila (queue) são especializações de Listas
 - Essencialmente, listas mais limitadas
- **Pilha (stack)**
 - TAD que representa uma pilha de elementos
 - O primeiro elemento a ser inserido é o último que será removido
 - "O primeiro a entrar será o último a sair"
 - FILO → First-In, Last-Out
 - Um pilha (stack) permite inserir e remover no **topo**
 - Somente é possível acessar o elemento que está no **topo**.



Principais Operações

- Principais operações

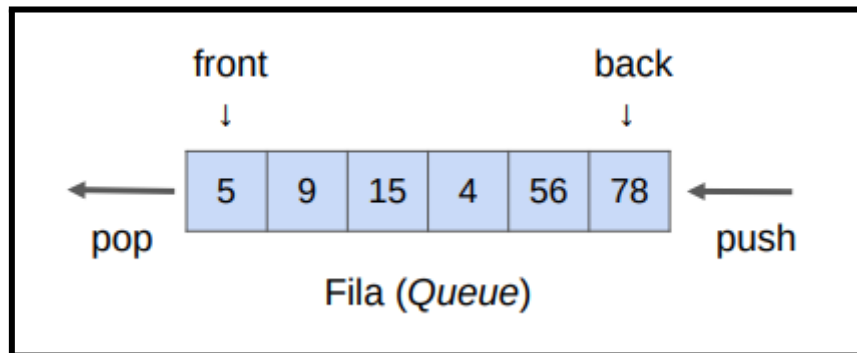
- | | | |
|---------------------|-----------------------------------|-----------------------------------|
| ○ Criar vazia | <code>stack<int> stk</code> | |
| ○ Copiar pilhas | <code>stk2 = stk</code> | |
| ○ Obter tamanho | <code>stk.size()</code> | |
| ○ Verificar vazia | <code>stk.empty()</code> | |
| ○ Obter topo | <code>stk.top()</code> | |
| ○ Adicionar ao topo | <code>stk.push(valor)</code> | <code><< empilhar</code> |
| ○ Remover do topo | <code>stk.pop()</code> | <code><< desempilhar</code> |



Print do Stack

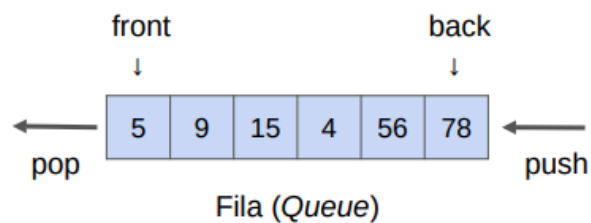
```
stack<int> stk;
stk.push(10);
stk.push(20);
stk.push(30);
while (!stk.empty()) {
    cout << stk.top() << " ";
    stk.pop();
}
```

Classe Queue - Fila



Fila de Elementos

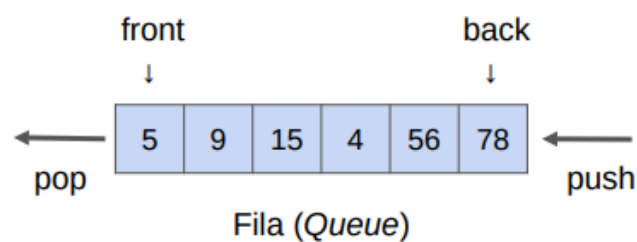
- **Fila (queue)**
 - TAD que representa uma fila de elementos
 - O primeiro elemento a ser inserido é o primeiro que será removido
 - "O primeiro a entrar será o primeiro a sair"
 - FIFO → First-In, First-Out
 - Um fila (queue) permite inserir elemento no **final** e remover do **início**
 - Somente é possível acessar o **primeiro** e o **último** elementos.



Principais Operações

- Principais operações

- | | | |
|----------------------|-----------------------------------|------------------|
| ○ Criar vazia | <code>queue<int> que</code> | |
| ○ Copiar filas | <code>que2 = que</code> | |
| ○ Obter tamanho | <code>que.size()</code> | |
| ○ Verificar vazia | <code>que.empty()</code> | |
| ○ Obter primeiro | <code>que.front()</code> | |
| ○ Obter último | <code>que.back()</code> | |
| ○ Adicionar ao final | <code>que.push(valor)</code> | << enfileirar |
| ○ Remover do início | <code>que.pop()</code> | << desenfileirar |



STL: Queue & Stack

```
int main() {  
    queue<int> que1;  
    que1.push(4);  
    que1.push(3);  
    que1.push(2);  
    que1.push(1);  
    print_queue(que1);  
  
    stack<int> stk1;  
    stk1.push(4);  
    stk1.push(3);  
    stk1.push(2);  
    stk1.push(1);  
    print_stack(stk1);  
    return 0;  
}  
  
void print_queue(queue<int> q) {  
    while (!q.empty()) {  
        cout << " " << q.front();  
        q.pop();  
    }  
    cout << '\n';  
}  
  
void print_stack(stack<int> q) {  
    while (!q.empty()) {  
        cout << " " << q.top();  
        q.pop();  
    }  
    cout << '\n';  
}
```

Por definição, não é possível percorrer os elementos de uma **queue** ou **stack**. Para imprimi-la, precisamos criar uma cópia e remover todos os seus elementos. Note que **não foi usado o operador de referência**.

Busca e Ordenação

Algoritmos de Busca

Busca Linear

A **busca linear** é um algoritmo de **pesquisa sequencial**, onde **começamos de uma extremidade e verificamos todos os elementos da lista até que o elemento desejado seja encontrado**. É o algoritmo de pesquisa mais simples.

Ou seja, procura linearmente um elemento, **verificando todas as posições do vetor**.

<https://www.programiz.com/dsa/linear-search>

Exemplo:

```
// Linear Search in C++
#include <iostream>
using namespace std;

int search(int array[], int n, int x) {
    // Going through array sequentially
    for (int i = 0; i < n; i++)
        if (array[i] == x)
            return i;
    return -1;
}

int main() {
    int array[] = {2, 4, 0, 1, 9};
    int x = 1;
    int n = sizeof(array) / sizeof(array[0]);

    int result = search(array, n, x);

    (result == -1) ? cout << "Element not found" : cout << "Element found
at index: " << result;
}
```

Complexidades da Busca Linear:

Tipo	O que significa
Time Complexity $O(n)$	Leva tempo proporcional ao tamanho da lista.
Space Complexity $O(1)$	Usa sempre a mesma quantidade de memória.

Obs: esse “O” faz parte da **Notação Big-O**, que é uma **forma de descrever a eficiência de um algoritmo em termos de tempo** (velocidade) ou **espaço** (memória) que ele consome, **conforme o tamanho da entrada (n) cresce**.

Aplicações

Para operações de pesquisa em arrays menores (<100 itens).

Busca Binária

- **Possível somente se o vetor está ordenado**
- Divide o vetor sucessivamente em duas metades, sendo que, após cada separação, procura o elemento somente na metade pertinente.
- Consegue encontrar um elemento com, no máximo, **$\log_2 N$ comparações**.
- <https://www.programiz.com/dsa/binary-search>

Exemplo:

```
// Binary Search in C++

#include <iostream>
using namespace std;

int binarySearch(int array[], int x, int low, int high) {

    // Repeat until the pointers low and high meet each other
    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (x == array[mid])
            return mid;

        if (x > array[mid])
            low = mid + 1;
    }
}
```

```
    else
        high = mid - 1;
    }

    return -1;
}
```

Complexidades

Tipo	O que significa
Best Case: $O(1)$	O valor é encontrado logo na primeira tentativa (está no meio da lista).
Average Case: $O(\log n)$	Em média, a busca é rápida porque corta a lista pela metade a cada passo.
Worst Case: $O(\log n)$	Mesmo no pior caso, o número de comparações é bem pequeno (logarítmico).
Space Complexity: $O(1)$	Usa memória constante — não precisa criar estruturas novas.

Aplicações

- Nas bibliotecas de Java, .Net, C++ STL
- Durante a depuração, a pesquisa binária é usada para identificar o local onde o erro aconteceu.

Busca Binária na STL

- Função `binary_search()` => lib `algorithm`
- `binary_search(primeiro_elemento, final_lista, valor)`

```
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
    int v[] = {1,4,6,9,10,24,45,87,100,122};
    bool found = binary_search(v, v+10, 100);
    if (found)
        cout << "Elemento encontrado" << endl;
    else
        cout << "Elemento nao encontrado" << endl;
    return 0;
}
```

Busca Binária em vector na STL:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {1,4,6,9,10,24,45,87,100,122};

    int found = binary_search(v.begin(), v.end(), 100);
    if (found)
        cout << "Elemento encontrado" << endl;
    else
        cout << "Elemento nao encontrado" << endl;
    return 0;
}
```

Algoritmos de Ordenação

- Ordenação é um problema e pode ser custoso
- Existem diversos algoritmos de ordenação
 - Dependendo dos dados, uma estratégia pode ser melhor que outra

- Vamos ver 3 desses algoritmos:
 - **Selection Sort** - <https://www.programiz.com/dsa/selection-sort>
 - **Bubble Sort** - <https://www.programiz.com/dsa/bubble-sort>
 - **Insertion Sort** - <https://www.programiz.com/dsa/insertion-sort>
- Depois, vamos usar a ordenação por meio da STL

Selection Sort

Selection Sort é um algoritmo de ordenação que **seleciona o menor elemento de uma lista não ordenada** em cada iteração e **coloca esse elemento no início da lista não ordenada**.

Como funciona?

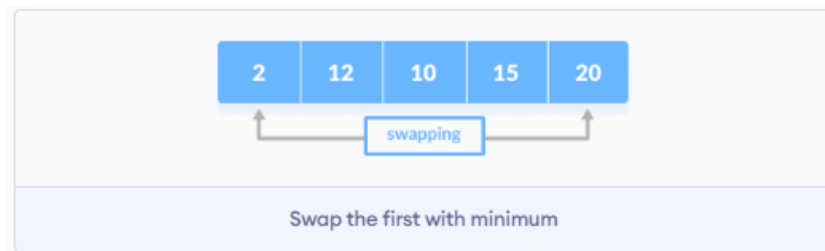
1. Defina o primeiro elemento como “**menor**”.



2. Compare o “menor” com o segundo elemento. **Se o segundo elemento for menor que o “menor”, atribua o segundo elemento como “menor”**. Compare o “menor” com o terceiro elemento. Novamente, se o terceiro elemento for menor, atribua o “menor” ao terceiro elemento, caso contrário, não faça nada. **O processo continua até o último elemento.**

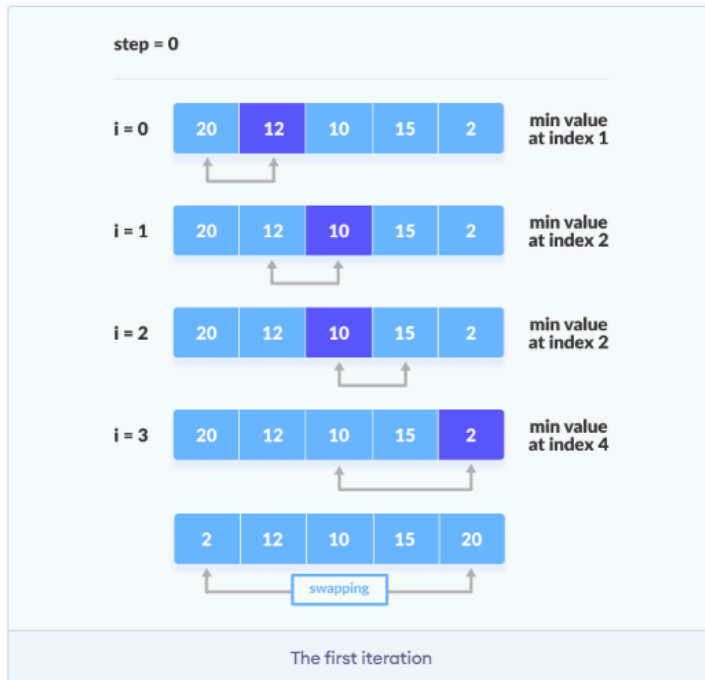


3. Após cada iteração, o “**menor**” é colocado na frente da lista não ordenada.

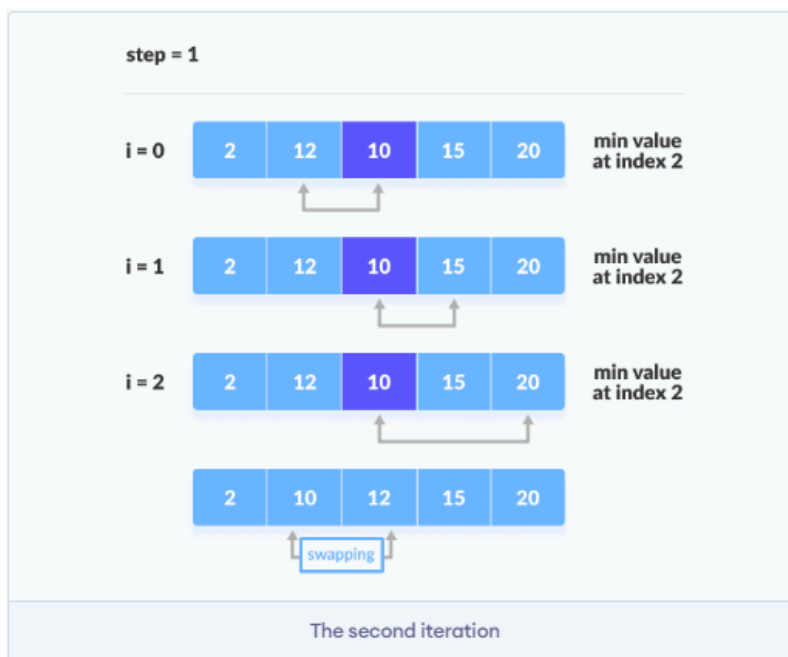


a.

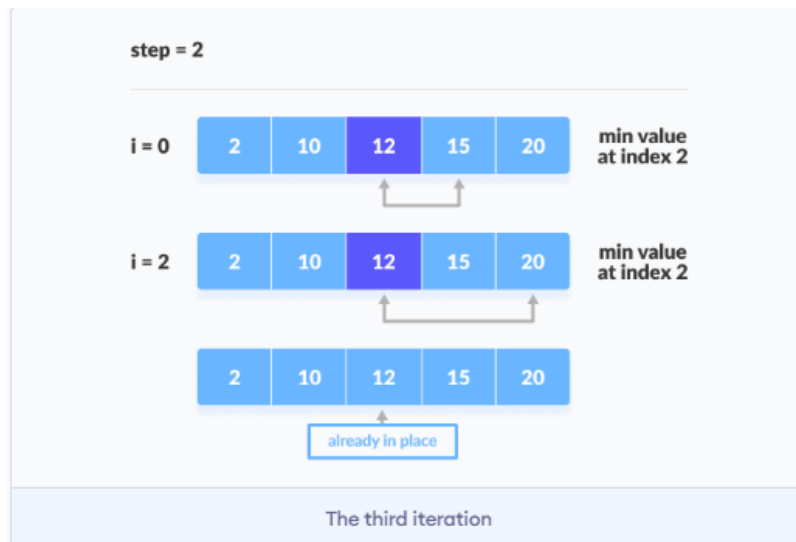
4. Para cada iteração, a **indexação começa no primeiro elemento não ordenado. A etapa 1 a 3 é repetida até que todos os elementos sejam colocados em suas posições corretas.**



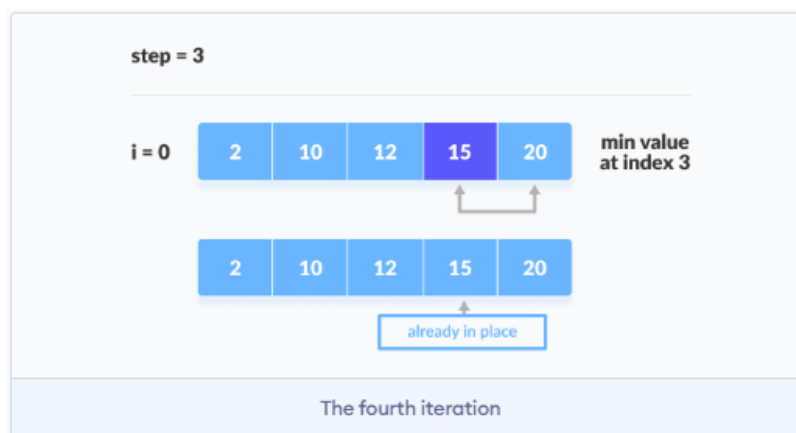
a.



b.



C.



d.

Exemplo:

```
void selectionSort(int array[], int size) {
    for (int step = 0; step < size - 1; step++) {
        int min_idx = step;
        for (int i = step + 1; i < size; i++) {

            // To sort in descending order, change > to < in this line.
            // Select the minimum element in each loop.
            if (array[i] < array[min_idx])
                min_idx = i;
        }

        // put min at the correct position
        swap(&array[min_idx], &array[step]);
    }
}
```

Complexidades

Complexidade	Descrição
Melhor Caso	O(n²) -> ocorre quando o array já está ordenado
Pior Caso	O(n²) -> Ocorre quando o array está em ordem inversa (decrecente para ordenação crescente)
Caso Médio	O(n²) -> Elementos estão em ordem aleatória
Complexidade de Espaço	O(1) -> Uso de apenas uma variável auxiliar (ex: min_idx)
Estabilidade	Não estável -> A troca de elementos pode alterar a ordem relativa de valores iguais

Número de comparações: $(n - 1) + (n - 2) + (n - 3) + \dots + 1 = n(n - 1) / 2$
quase igual a n^2 .

Complexidade = O(n²)

Além disso, podemos analisar a complexidade simplesmente observando o número de loops. Existem 2 loops, então a complexidade é $n \cdot n = n^2$.

Aplicações

- Listas pequenas para serem ordenadas;
- Custo de troca irrelevante;
- Verificação total obrigatória;
- Memória com escrita limitada, ideal para dispositivos como flash memory (cada escrita reduz a vida útil), pois minimiza trocas (ex.: vs Bubble Sort, que faz O(n²) trocas).

Bubble Sort

O **Bubble Sort** é um algoritmo de ordenação que **compara dois elementos adjacentes e os troca até que estejam na ordem pretendida**.

Assim como o movimento das bolhas de ar na água que sobem à superfície, cada elemento da matriz se move para o final em cada iteração. Portanto, é chamado de bubble sort.

Como funciona?

Suponha que estamos tentando ordenar os elementos em ordem crescente.

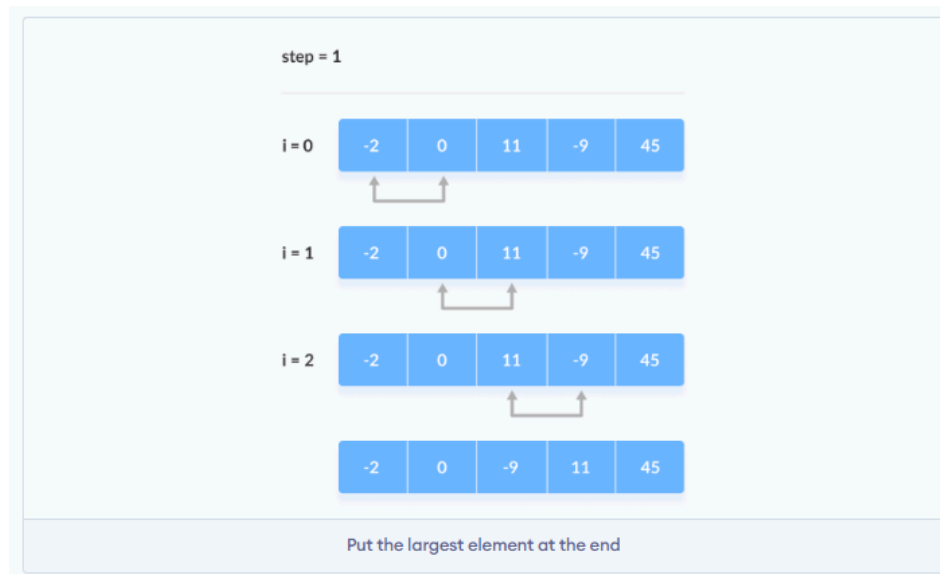
1. Primeira iteração (compare e troca)

- A partir do **primeiro índice, compare o primeiro e o segundo elementos**.
- Se o primeiro elemento for **maior** que o segundo elemento, **eles serão trocados**.
- Agora, **compare o segundo e o terceiro elementos**. Troque-os se não estiverem em ordem.
- O processo acima continua até o último elemento**.



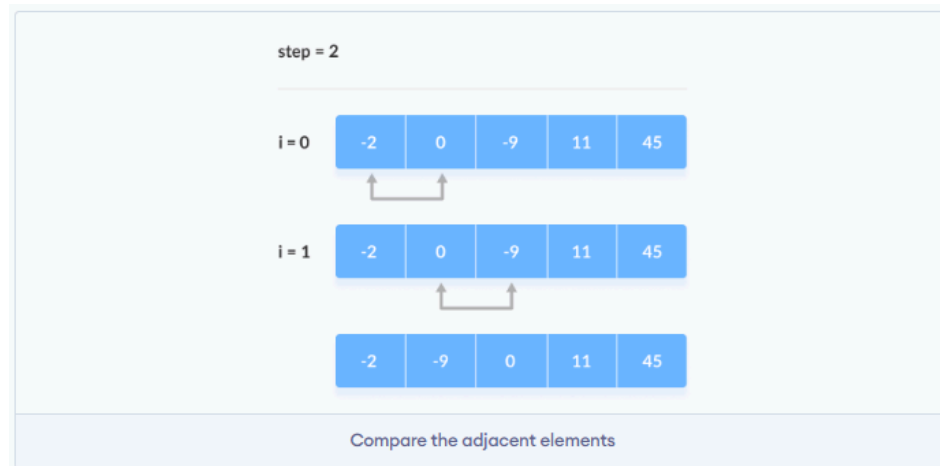
2. Iterações restantes

- O mesmo processo continua para as **iterações restantes**.
- Após cada iteração, **o maior elemento entre os elementos não ordenados é colocado no final**.



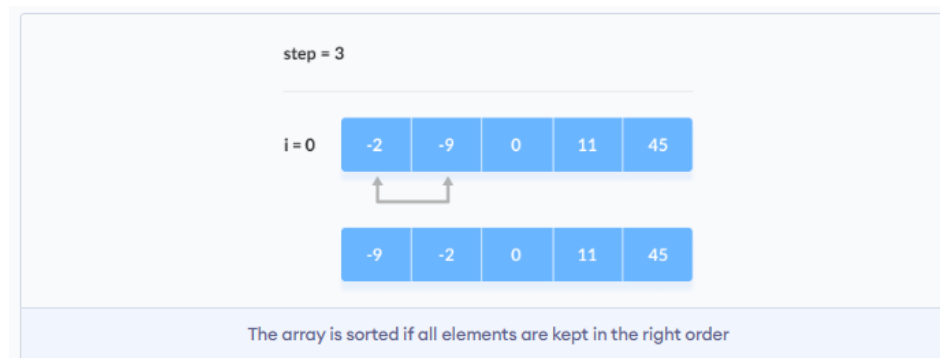
c.

d. Em cada iteração, a **comparação ocorre até o último elemento não ordenado**.



e.

f. O array é ordenado quando todos os elementos não ordenados são colocados em suas posições corretas.



g.

Exemplo:

```
// perform bubble sort
void bubbleSort(int array[], int size) {

    // loop to access each array element
    for (int step = 0; step < size - 1; ++step) {

        // loop to compare array elements
        for (int i = 0; i < size - step - 1; ++i) {

            // compare two adjacent elements
            // change > to < to sort in descending order
            if (array[i] > array[i + 1]) {

                // swapping elements if elements
                // are not in the intended order
                int temp = array[i];
                array[i] = array[i + 1];
                array[i + 1] = temp;
            }
        }
    }
}
```

Bubble Sort Otimizado

- No algoritmo acima, todas as comparações são feitas mesmo que o array já esteja ordenado.
- **Isso aumenta o tempo de execução.**
- Para resolver isso, podemos introduzir uma variável extra **swapped**. O **valor de swapped é definido como true se ocorrer troca de elementos**. Caso contrário, ele será definido como **falso**.
- Após uma iteração, se não houver troca, o valor de **swapped** será **falso**. **Isso significa que os elementos já estão ordenados e não há necessidade de realizar mais iterações.**
- Isso reduzirá o tempo de execução e ajudará a otimizar o bubble sort.

```
// perform bubble sort
void bubbleSort(int array[], int size) {

    // loop to access each array element
    for (int step = 0; step < (size-1); ++step) {

        // check if swapping occurs
        int swapped = 0;

        // loop to compare two elements
        for (int i = 0; i < (size-step-1); ++i) {

            // compare two array elements
            // change > to < to sort in descending order
            if (array[i] > array[i + 1]) {

                // swapping occurs if elements
                // are not in intended order
                int temp = array[i];
                array[i] = array[i + 1];
                array[i + 1] = temp;

                swapped = 1;
            }
        }

        // no swapping means the array is already sorted
        // so no need of further comparison
        if (swapped == 0)
            break;
    }
}
```

Complexidades

Time Complexity	
Best	$O(n)$
Worst	$O(n^2)$
Average	$O(n^2)$
Space Complexity	
$O(1)$	
Stability	
Yes	

Aplicações

Bubble sort é usado se:

- a complexidade não importa
- código curto e simples é preferido

Insertion Sort

O Insertion Sort é um algoritmo de ordenação que coloca um elemento não ordenado em seu local adequado em cada iteração.

O Insertion Sort funciona de forma semelhante à ordenação de cartas em nossa mão em um jogo de cartas.

Assumimos que a primeira carta já está ordenada e, em seguida, selecionamos uma carta não ordenada. Se a carta não ordenada for maior que a carta na mão, ela será colocada à direita, caso contrário, à esquerda. Da mesma forma, outras cartas não ordenadas são retiradas e colocadas em seus devidos lugares.

Uma abordagem semelhante é usada pelo Insertion Sort.

Como funciona?

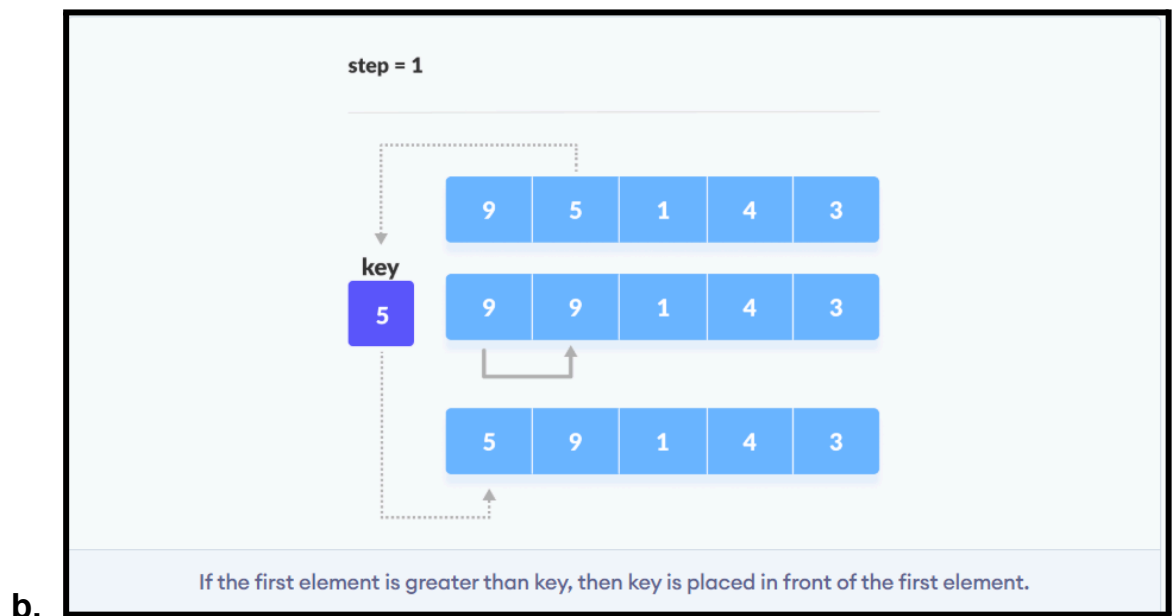
Suponha que precisamos classificar o seguinte array.



1. Presume-se que o primeiro elemento do array esteja ordenado.

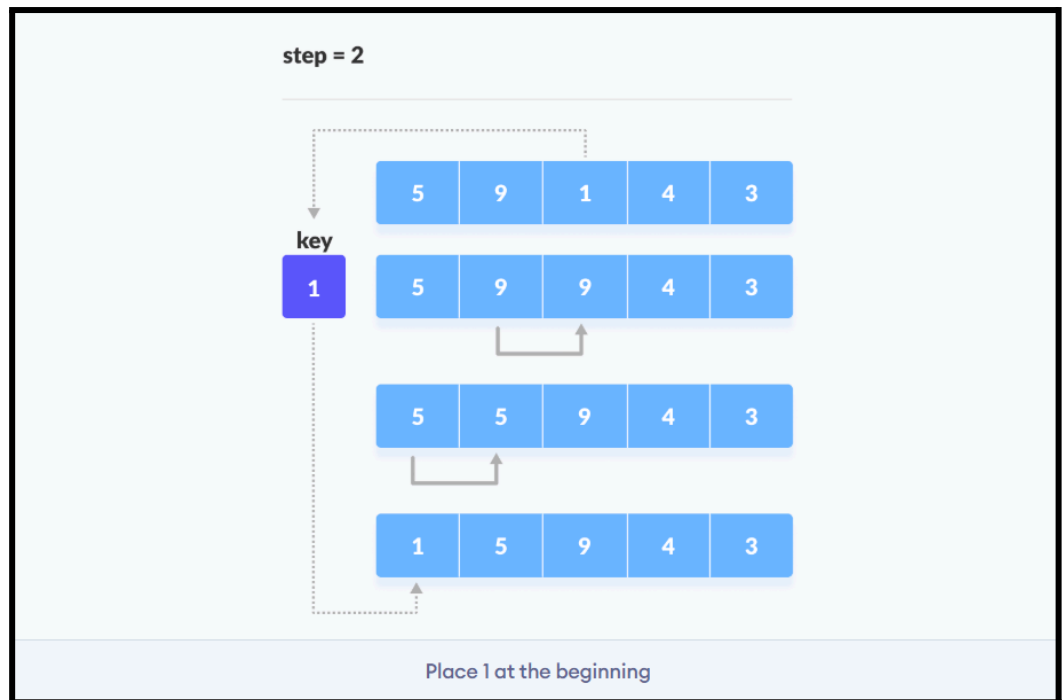
Pegue o **segundo elemento** e **guarde-o separadamente no "key"**.

- Compare o key com o primeiro elemento.** Se o primeiro elemento for maior que o key, o key será colocado na frente do primeiro elemento.



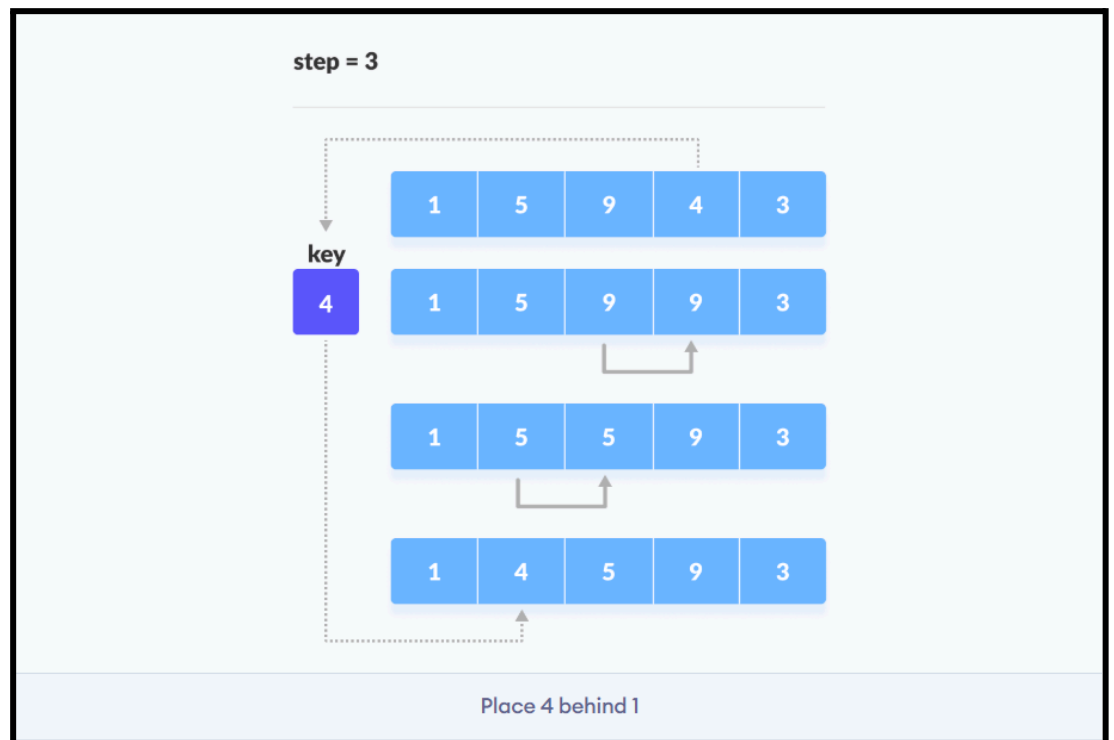
2. Agora, os dois primeiros elementos estão ordenados.

- Pegue o **terceiro elemento** e compare-o com os elementos à esquerda. Coloque-o logo atrás do elemento menor que ele. Se não houver elemento menor que ele, coloque-o no início do array.

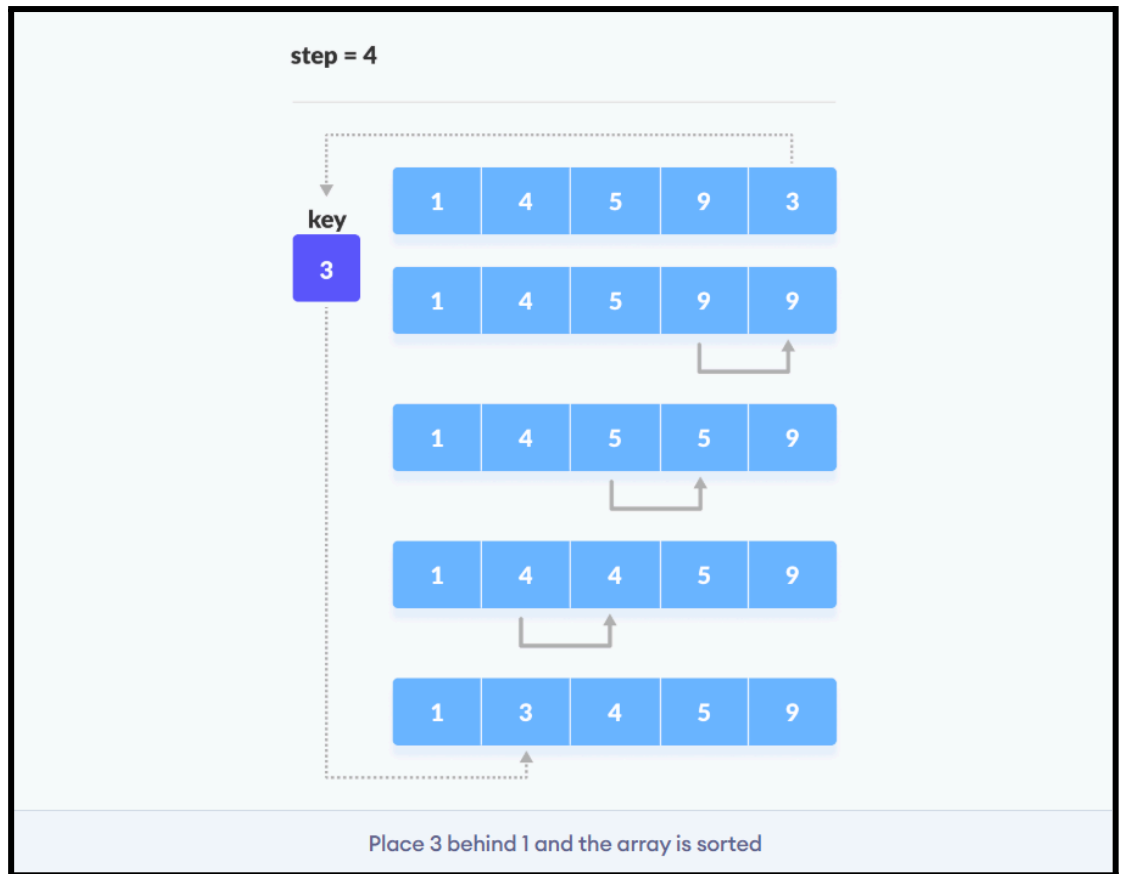


b.

3. Da mesma forma, coloque todos os elementos não ordenados em sua posição correta.



a.



b.

Exemplo:

```
void insertionSort(int array[], int size) {
    for (int step = 1; step < size; step++) {
        int key = array[step];
        int j = step - 1;

        // Compare key with each element on the left of it until an element
        // smaller than
        // it is found.
        // For descending order, change key < array[j] to key > array[j].
        while (j >= 0 && key < array[j]) {
            array[j + 1] = array[j];
            --j;
        }
        array[j + 1] = key;
    }
}
```

Complexidades

Time Complexity	
Best	$O(n)$
Worst	$O(n^2)$
Average	$O(n^2)$
Space Complexity	
	$O(1)$
Stability	
	Yes

Aplicações

O **Insertion Sort** é usado quando:

- O array tem um pequeno número de elementos
- Restam apenas alguns elementos a serem ordenados

Ordenação em STL → `sort()`

- Ordenação na STL: **função `sort()` => lib `<algorithm>`**
 - **`sort(primeiro_elemento, após_final, função_critério)`**

```
int main() {  
    int v1[10] = {5,4,3,2,3,4,1,7,5,5};  
    string v2[10] = {"Xayane", "Carla", "Ana", "Joe", "Bane"};  
  
    sort(v1, v1+10);  
    printArray(10, v1);  
  
    sort(v2, v2+10);  
    printArray(10, v2);  
  
    return 0;  
}
```

- **Ordenação em vector**

```
bool desc_order(int elem, int next) { // "true" define o critério
    return elem > next;
}

int main() {
    vector<int> vec1 = {5,4,3,2,3,4,1,7,5,5};
    vector<string> vec2 = {"Xayane", "Carla", "Ana", "Joe", "Bane"};

    sort(vec1.begin(), vec1.end(), desc_order);    // ordenação decrescente
    printVector(vec1);

    sort(vec2.begin(), vec2.end());
    printVector(vec2);
    return 0;
}
```

○

- **Ordenação em forward_list e list**

```
bool desc_order(int elem, int next) { // "true" define o critério
    return elem > next;
}

int main() {
    forward_list<int> list1 = { 5,4,3,2,3,4,1,7,5,5 }; // lista encadeada
    list1.sort(desc_order); // ordem decrescente

    // percorrendo e imprimindo usando "for each" (range-based for)
    for (int elem : list1) {
        cout << elem << ' ';
    }
    cout << endl;
    return 0;
}
```

○

- **Ordenação em coleções de dados (structs)**

```
bool asc_order_entity(Entity elem, Entity next) {
    return elem.id < next.id;
}

int main() {
    forward_list<Entity> list1;
    for (int i = 1; i <= 10; i++) {
        Entity ent = {i, "John_Doe_" + to_string(i)}; // ex: {1, John_Doe_1}
        list1.push_front(ent); // adiciona ao início da lista (ficará decrescente)
    }
    list1.sort(asc_order_entity); // ordena crescentemente por ID da entidade

    for (Entity elem : list1) {
        cout << elem.id << ", " << elem.name << endl;
    }
    return 0;
}
```

○

Recursividade

Recursividade é uma técnica de programação onde uma **função chama a si mesma** direta ou indiretamente para resolver um problema.

A ideia é **dividir um problema grande em problemas menores** (do mesmo tipo) até chegar em um caso base simples que pode ser resolvido sem recursão.

(Lembrando, não se pode usar for e while!)

Estrutura básica de uma função recursiva

Uma função recursiva geralmente tem **dois componentes essenciais**:

1. **Caso base** → **Condição que encerra a recursão** (evita loop infinito).
2. **Chamada recursiva** → **A função se chama novamente com um problema menor.**

Exemplo simples (fatorial):

```
#include <iostream>
using namespace std;

int fatorial(int n) {
    if (n == 0 || n == 1) { // Caso base
        return 1;
    }
    return n * fatorial(n - 1); // Chamada recursiva
}

int main() {
    cout << "Fatorial de 5 = " << fatorial(5) << endl;
    return 0;
}
```

Tipos de Recursividade

Existem vários tipos de recursividade, mas os principais são:

1. Recursão Direta

A função chama a si mesma diretamente.

- Exemplo: **cálculo do fatorial** (acima).

2. Recursão Indireta

Uma função chama outra função, que depois chama a primeira de volta.

- Exemplo:

```
#include <iostream>
using namespace std;

void funcA(int n);
void funcB(int n);

void funcA(int n) {
    if (n > 0) {
        cout << n << " ";
        funcB(n - 1); // Chama B
    }
}

void funcB(int n) {
    if (n > 1) {
        cout << n << " ";
        funcA(n / 2); // Chama A
    }
}

int main() {
    funcA(10);
    return 0;
}
```

3. Recursão em Cauda

A chamada recursiva é a **última instrução** da função.

Isso permite que o compilador otimize a memória (sem empilhar chamadas extras).

- Exemplo:

```
#include <iostream>
using namespace std;

int soma(int n, int acumulador = 0) {
    if (n == 0) return acumulador; // Caso base
    return soma(n - 1, acumulador + n); // Recursão em cauda
}

int main() {
    cout << "Soma de 1 até 5 = " << soma(5) << endl;
    return 0;
}
```

4. Recursão Não em Cauda

A chamada recursiva **não é a última operação** da função.

- Exemplo: **cálculo de potência**

```
#include <iostream>
using namespace std;

int potencia(int base, int exp) {
    if (exp == 0) return 1; // Caso base
    return base * potencia(base, exp - 1); // Não é tail recursion
}

int main() {
    cout << "2^5 = " << potencia(2, 5) << endl;
    return 0;
}
```

(**Não é em cauda**, pois após o último retorno da função potência, ainda tem que fazer a multiplicação da base * esse retorno!)

5. Recursão Aninhada

A função chama a si mesma dentro de outra chamada recursiva.

- Exemplo:

```
#include <iostream>
using namespace std;

int func(int n) {
    if (n > 100) return n - 10;
    return func(func(n + 11)); // Chamada recursiva dentro de outra
}

int main() {
    cout << func(95) << endl;
    return 0;
}
```

Quando usar?

- Problemas que podem ser naturalmente divididos em subproblemas menores (ex: fatorial, Fibonacci, busca binária, árvores).
- Quando a solução fica mais simples e legível do que com loops.
- Mas cuidado: recursão mal usada pode causar estouro de pilha (stack overflow).